



IT GENIUS
THINK IT • BE GENIUS

Python Basic 2026

ปูพื้นฐานการเขียนโปรแกรมภาษา Python
ตั้งแต่เริ่มต้นจนถึงการประยุกต์ใช้งานจริง

Beginner Course • Hands-on Workshop • 2 Days (12 Hours)

Python 3.13/3.14 • uv • VS Code • PyCharm



พื้นฐานแน่น
เข้าใจง่าย



ลงมือทำจริง
เรียนรู้จากการปฏิบัติ



ต่อยอดได้จริง
ใช้งานได้ทันที

```
main.py
1 # เริ่มต้นกับ Python
2 name = "IT Genius"
3 message = f"Hello, {name}!"
4 print(message)
5
6 for i in range(1, 6):
7     print(f"Count: {i}")
```

```
$ uv init my-project
$ cd my-project
$ uv run main.py
Hello, Python!
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```



เรียนจบ
นำไปใช้งานจริง
ได้ทันที!



วิทยากร



อ.สามิตร ไกยม (ปาน)

ปริญญาโท คณะเทคโนโลยีและสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าพระนครเหนือ

IT Genius Engineering Co., Ltd.

Frontend

Angular, React, Vue, Next, Nuxt, Bootstrap, Tailwind CSS

Backend

PHP, Python, Java, Kotlin, Go, Rust, NodeJS, NestJS, .NET

Database

MySQL, MariaDB, PostgreSQL, MS SQL Server, MongoDB

Mobile

Java, Kotlin, Swift, KMP, Flutter, React Native, Expo

DevOps & Monitoring

Git, Docker, Kubernetes, Jenkins, CI/CD, Sentry, Grafana

ก่อนเริ่มอบรม

แบบทดสอบก่อนอบรม (Pre-test)

เข้าทำแบบทดสอบที่

wayground.com/join

กรอก Join Code ที่วิทยากรแจ้งหน้าห้อง แล้วเริ่มทำแบบทดสอบได้ทันที



SCAN ME

COURSE MATERIALS

ดาวน์โหลดเอกสารประกอบการอบรม

เอกสารประกอบการอบรม ไฟล์ตัวอย่างโค้ดทุก Workshop และไฟล์ข้อมูลที่ใช้ตลอดหลักสูตร รวมอยู่ใน GitHub Repository ของหลักสูตร

GitHub Repository

github.com/iamsamitdev/basic-python-2026

วิธีที่ 1 — ดาวน์โหลดเป็นไฟล์ ZIP

- 1 เปิดลิงก์ Repository ด้านบนในเบราว์เซอร์
- 2 กดปุ่ม Code สีเขียว แล้วเลือก Download ZIP
- 3 แยกไฟล์ไว้ที่ C:\PythonTraining

วิธีที่ 2 — Clone ด้วย Git (แนะนำ)

```
cd C:\PythonTraining
git clone https://github.com/iamsamitdev/
    basic-python-2026.git
cd basic-python-2026
```

ถ้ายังไม่มี Git ในเครื่อง ให้ใช้วิธีที่ 1 ก่อน แล้วเราจะติดตั้ง Git พร้อมกันในวันที่ 2



IT GENIUS
THINK IT • BE GENIUS

Python Basic 2026

ปูพื้นฐานการเขียนโปรแกรมภาษา Python
ตั้งแต่เริ่มต้นจนถึงการประยุกต์ใช้งานจริง

Beginner Course • Hands-on Workshop • 2 Days (12 Hours)

Python 3.13/3.14 • uv • VS Code • PyCharm



พื้นฐานแน่น
เข้าใจง่าย



ลงมือทำจริง
เรียนรู้จากการปฏิบัติ



ต่อยอดได้จริง
ใช้งานได้ทันที

DAY

1

```
main.py
1 # เริ่มต้นกับ Python
2 name = "IT Genius"
3 message = f"Hello, {name}!"
4 print(message)
5
6 for i in range(1, 6):
7     print(f"Count: {i}")
```

```
$ uv init my-project
$ cd my-project
$ uv run main.py
Hello, Python!
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```



เรียนจบ
นำไปใช้งานจริง
ได้ทันที!

COURSE OVERVIEW

ภาพรวมหลักสูตร Python Basic 2026

ระยะเวลา

2 วัน

12 ชั่วโมง

ระดับ

Beginner

ไม่ต้องมีพื้นฐาน

รูปแบบ

Hands-on

บรรยายพร้อมปฏิบัติจริง

เวลาอบรม

09:00-16:00

พัก 12:00-13:00

เนื้อหาฉบับปี 2026 ปรับให้ตรงกับเครื่องมือที่ใช้งานจริง โดยใช้ **uv** เป็นเครื่องมือหลักตลอดหลักสูตร ควบคู่กับ Visual Studio Code และ PyCharm พร้อมการเขียนโค้ดสไตล์สมัยใหม่ด้วย f-string และ Type Hints

LEARNING OBJECTIVES

เมื่อจบวันที่ 1 ท่านจะทำได้

01

อธิบายจุดเด่น ข้อจำกัด และขอบเขตการใช้งาน Python ในปี 2026 ได้

02

ติดตั้ง uv สลับเวอร์ชัน Python และรันโปรแกรมด้วย `uv run` ได้เอง

03

ตั้งค่า VS Code ให้ชี้ Interpreter ไปยัง `.venv` ของ uv ได้ถูกต้อง

04

เขียนโปรแกรมด้วยตัวแปร ชนิดข้อมูล เงื่อนไข และการทำซ้ำได้

05

เลือกใช้ List, Tuple, Set, Dictionary ให้เหมาะสมกับปัญหา

06

สร้างฟังก์ชันที่รับค่าได้หลายรูปแบบ พร้อม Docstring และ Type Hints

07

แยกโค้ดเป็นโมดูล ติดตั้งไลบรารีด้วย uv และส่งต่อโปรเจกต์ให้เพื่อนได้

08

สร้างโปรเจกต์ PyShop Mini โครงร่างแรกที่จัดการรายการสินค้าได้จริง

AGENDA

กำหนดการวันที่ 1

09:00-09:30	ทดสอบก่อนเรียน (Pretest) และแนะนำหลักสูตร
09:30-10:15	Module 1 แนะนำภาษา Python (Overview)
10:15-12:00	Module 2 เตรียมเครื่องมือและสภาพแวดล้อมด้วย uv
12:00-13:00	พักรกลางวัน
13:00-14:15	Module 3 พื้นฐานภาษา Python
14:15-15:00	Module 4 ชนิดข้อมูลแบบกลุ่ม (Collections)
15:00-15:30	Module 5 ฟังก์ชัน (Function)
15:30-16:00	Module 6 Module และ Package + Workshop Day 1

COURSE PROJECT

PyShop Mini — โปรแกรมที่เราจะสร้างด้วยกัน

ระบบจัดการสินค้าคงคลังร้านค้าขนาดเล็ก (ข้อมูลจำลองทั้งหมด) เราจะค่อย ๆ พัฒนาโปรแกรมเดี่ยวนี้นี่ต่อเนื่องตลอด 2 วัน
แผนการเขียนตัวอย่างสั้น ๆ ที่ไม่ต่อกัน

DAY 1

PyShop Mini v1

โปรแกรมแบบ Console ที่มีเมนู ค้นหา เพิ่มสินค้า ปรับสต็อก และ
รายงานสรุป โดยแยกโค้ดเป็นโมดูลตามหน้าที่

DAY 2

PyShop Mini v2

ต่อยอดเป็นไฟล์ CSV/JSON, OOP, ฐานข้อมูล PostgreSQL และ
หน้าจอ GUI จริง

01

MODULE 01 · 09:30-10:15

แนะนำภาษา Python

จุดเด่น ข้อจำกัด ขอบเขตการใช้งานจริง เส้นทางอาชีพ และการเลือกเวอร์ชัน



ทำไม Python เหมาะกับการเริ่มต้นเขียนโปรแกรม

C (7 บรรทัด)

```
#include <stdio.h>
int main() {
    for (int i=1; i<=5; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Python (2 บรรทัด)

```
for i in range(1, 6):
    print(i)
```

- ✓ ไม่ต้องประกาศชนิดข้อมูล เริ่มเขียนได้ทันที
- ✓ ไม่มีปีกกา ใช้การย่อหน้าแทน โค้ดถูกบังคับให้อ่านง่าย
- ✓ ไม่ต้อง Compile แก่แล้วรันเห็นผลทันที รอบการเรียนรู้สั้น
- ✓ ไลบรารีมหาศาลบน PyPI ทำงานจริงได้เร็ว
- ✓ ชุมชนใหญ่ เอกสารเยอะ ค้นหาคำตอบง่าย

ปรัชญาของ Python คือ **"โค้ดถูกอ่านบ่อยกว่าถูกเขียน"** วยากรณ์จึงใกล้เคียงภาษาอังกฤษ

จุดเด่นและข้อจำกัดที่ต้องรู้

จุดเด่น

เขียนเร็ว อ่านง่าย เรียนรู้ไว
ทำงานข้ามระบบปฏิบัติการ
ไลบรารีครอบคลุมเกือบทุกงาน
เป็นภาษาหลักของสาย Data และ AI
เหมาะกับงาน Automation และ Prototype

ข้อจำกัด

ทำงานช้ากว่าภาษาคอมไพล์ในงานคำนวณล้วน
ต้องมี Python Runtime ในเครื่องปลายทาง
ใช้หน่วยความจำมากกว่าภาษาระดับล่าง
GIL ทำให้ Thread เดียวกันไม่ประมวลผลขนานเต็มที่
ไม่เหมาะกับงาน Real-time และ Mobile App โดยตรง

งาน Data และ AI ที่ใช้ Python เร็วได้เพราะไลบรารีอย่าง NumPy และ Pandas เขียนแกนกลางด้วย C ส่วน Python ทำหน้าที่เป็นตัวส่งงาน การเลือกไลบรารีที่เหมาะสมจึงสำคัญกว่าการปรับโค้ดทีละบรรทัด

Python ไปใช้งานจริงที่ไหน และต่อยอดไปทางไหนได้

Web Application

ระบบหลังบ้าน, REST API

Django · Flask · FastAPI

Automation

รวมไฟล์ Excel รายเดือน, ส่งอีเมลอัตโนมัติ

openpyxl · pandas · requests

Data Analytics

ทำความสะอาดข้อมูล, สรุปยอดขาย

pandas · matplotlib · plotly

AI / Machine Learning

ระบบแนะนำสินค้า, NLP, Computer Vision

scikit-learn · PyTorch

DevOps / Sys Admin

สคริปต์ Deploy, ตรวจสอบ Log, จัดการ Cloud

boto3 · paramiko · fabric

การศึกษา / งานวิจัย

สอนโปรแกรมมิ่ง, จำลองการคำนวณ

Jupyter · SymPy · SciPy

สำหรับคนทำงานทั่วไป งานกลุ่ม **Automation** ให้ผลตอบแทนเร็วที่สุด เช่น สคริปต์อ่านไฟล์ CSV หลายสิบไฟล์แล้วสรุปเป็นไฟล์เดียว ซึ่งเราจะทำในวันที่ 2

เวอร์ชันของ Python และความแตกต่างที่ควรทราบ

เวอร์ชัน	สถานะปี 2026	สิ่งที่ควรรู้
Python 2.7	หมดอายุแล้ว	เจอในโค้ดเก่าเท่านั้น จุดสังเกตคือ print ไม่มีวงเล็บ
Python 3.8-3.11	ยังพบบ่อยในระบบเดิม	ใช้งานได้ แต่ไม่ควรเริ่มโปรเจกต์ใหม่
Python 3.12	เสถียร ใช้แพร่หลาย	ข้อความ Error ชัดเจนขึ้นมาก
Python 3.13	แนะนำสำหรับหลักสูตรนี้	REPL ใหม่ที่ใช้งานสะดวก, ทดลอง free-threaded
Python 3.14	เวอร์ชันใหม่ล่าสุด	ใช้ได้ แต่ไลบรารีบางตัวอาจยังตามไม่ทัน

กฎการเลือกเวอร์ชันในงานจริง อย่าเลือกใหม่ที่สุดเสมอไป ให้เลือก **เวอร์ชันเสถียรที่ไลบรารีที่ต้องใช้รองรับครบแล้ว** โดยทั่วไปคือรุ่นที่ออกมาแล้วอย่างน้อย 6-12 เดือน

02

MODULE 02 · 10:15-12:00

เตรียมเครื่องมือและสภาพแวดล้อมด้วย nv

ปัญหาคลาสสิกของผู้เริ่มต้นไม่ใช่ไวยากรณ์ แต่คือ "เครื่องผมรันไม่ได้แต่เครื่องอาจารย์รันได้" nv เกิดมาเพื่อแก้ปัญหานี้โดยเฉพาะ



ติดตั้ง Python ลงเครื่องคอมพิวเตอร์

เริ่มจากติดตั้งตัวภาษา Python ให้เข้าใจภาพรวมของการติดตั้งแบบดั้งเดิมก่อน แล้วในหัวข้อต่อไปเราจะใช้ **uv** เข้ามาจัดการเวอร์ชันและสภาพแวดล้อมแทน

Windows

- 1 ดาวน์โหลดตัวติดตั้งจาก python.org/downloads เลือกรุ่น **Python 3.13**
- 2 **ติ๊ก Add python.exe to PATH** ที่หน้าจอแรกก่อนกด Install
- 3 เลือก Install Now แล้วรอจนขึ้น Setup was successful
- 4 ปิด Terminal แล้วเปิดใหม่ เพื่อให้ PATH ถูกโหลด

macOS

```
brew install python@3.13
```

ตรวจสอบว่าติดตั้งสำเร็จ

```
python --version
# Python 3.13.x

pip --version
# pip 25.x from ...\\site-packages\\pip
```

ถ้าพิมพ์ python แล้ว Microsoft Store เด้งขึ้นมา แปลว่ายังไม่ได้ติดตั้ง Python จริง หรือลืมติ๊ก Add to PATH ให้ติดตั้งซ้ำอีกครั้ง

ในทางปฏิบัติ uv ติดตั้งตัว Python ให้ได้เองด้วยคำสั่ง `uv python install` ท่านจึงไม่ต้องดาวน์โหลดจากเว็บอีกเมื่อขึ้นโปรเจกต์ใหม่

รู้จัก uv และเหตุผลที่หลักสูตรเลือกใช้

uv คือเครื่องมือจัดการ Python ยุคใหม่ พัฒนาโดยบริษัท Astral ทีมเดียวกับ Ruff เขียนด้วยภาษา Rust จุดขายคือ **เร็วมาก** และ **รวมงานหลายอย่างไว้ในคำสั่งเดียว**

วิธีดั้งเดิม — หลายเครื่องมือ

pyenv → ติดตั้ง Python
venv → สร้าง virtual env
pip → ติดตั้งไลบรารี
pip freeze → requirements.txt
pip-tools → ตรึงเวอร์ชัน

จำหลายคำสั่ง ซ้ำ ตกหล่นง่าย



วิธีใหม่ — เครื่องมือเดียว

```
uv python install uv init / uv venv uv add / uv  
sync / uv lock uv run uv export
```

คำสั่งเดียวจบ เร็วกว่ามาก

uv เทียบกับ pip + venv

ประเด็น	pip + venv (เดิม)	uv (ที่เราจะใช้)
ความเร็วติดตั้งแพ็คเกจ	ปกติ	เร็วกว่า 10-100 เท่าในหลายกรณี
ติดตั้งตัว Python เอง	ทำไม่ได้	ทำได้ในตัว
สร้าง venv	<code>python -m venv .venv</code>	<code>uv venv</code>
ล็อกเวอร์ชันไลบรารี	ต้องใช้ pip-tools เสริม	มี <code>uv.lock</code> ในตัว
ไฟล์มาตรฐานโปรเจกต์	<code>requirements.txt</code>	<code>pyproject.toml</code> (PEP 621)
ทำให้ทุกเครื่องเหมือนกัน	ยาก	<code>uv sync</code> คำสั่งเดียว

หลักสูตรนี้ใช้ uv เป็นหลัก แต่ยังสอน pip และ venv ให้รู้จักด้วย เพราะโค้ดและเอกสารเก่าจำนวนมากยังใช้อยู่ และท่านต้องอ่านออกเมื่อไปเจอในงานจริง

ติดตั้ง uv และเลือกเวอร์ชัน Python

Windows (PowerShell)

```
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"
```

macOS / Linux

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

ปัญหาที่พบบ่อยที่สุดในห้องอบรม ติดตั้งเสร็จแล้วพิมพ์ uv แล้วขึ้นว่า not recognized สาเหตุเกือบ 100% คือ **ยังไม่ได้ปิด-เปิด Terminal ใหม่**

uv --version	# ตรวจสอบว่าติดตั้งสำเร็จ
uv python list	# ดูเวอร์ชันที่ติดตั้งได้และที่มีอยู่
uv python install 3.13	# ติดตั้งเวอร์ชันที่โหลดหลักสุด
uv python install 3.12 3.14	# หลายเวอร์ชันในเครื่องเดียวได้ ไม่ติดกัน

uv แยกแต่ละเวอร์ชันไว้คนละที่ และให้แต่ละโปรเจกต์เลือกใช้เองผ่านไฟล์ .python-version โดยไม่ต้องแก้ PATH ของระบบ

สร้างโปรเจกต์ด้วย uv init และรู้จัก pyproject.toml

```
cd C:\PythonTraining
uv init pyshop --python 3.13
cd pyshop
```

```
pyshop/
├─ .git/
├─ .gitignore
├─ .python-version
├─ README.md
├─ main.py
└─ pyproject.toml
```

```
[project]
name = "pyshop"
version = "0.1.0"
description = "PyShop Mini"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []
```

name ชื่อโปรเจกต์ ใช้เมื่อเผยแพร่ขึ้น PyPI

requires-python เวอร์ชันขั้นต่ำที่โปรเจกต์ต้องการ

dependencies รายการไลบรารีที่ต้องใช้ ซึ่ง uv add จะมาเติมให้

uv init สร้าง git repository ให้อัตโนมัติ ซึ่งเราจะได้ใช้จริงใน Module 11 ของวันพรุ่งนี้

Virtual Environment คืออะไร และทำไมต้องมี

ไม่มี venv — ติดตั้งรวมทีเดียว

Python ของระบบ

requests 2.28 ← โปรเจกต์ A ใช้

requests 2.31 ← โปรเจกต์ B ใช้

หับกัน พังทั้งคู่

มี venv — แยกกล่องต่อโปรเจกต์

Project A/.venv → requests 2.28

Project B/.venv → requests 2.31

ต่างคนต่างอยู่ ลบทิ้งได้อิสระ

```
uv venv                                # สร้าง .venv ในโฟลเดอร์โปรเจกต์
uv venv --python 3.13                 # ระบุเวอร์ชันเฉพาะ

python -m venv .venv                  # วิธีเดิม ต้อง activate ทุกครั้งก่อนใช้งาน
.\.venv\Scripts\Activate.ps1
```

ถ้าใช้ `uv run` และ `uv add` ก่านไม่ต้อง activate เลย `uv` หา `.venv` ให้เอง

ห้ามเอา `.venv` ขึ้น Git เติดขาด ให้ส่ง `pyproject.toml` กับ `uv.lock` แทน แล้วปลายทางรัน `uv sync`

ตั้งค่า Visual Studio Code และ PyCharm

- 1 ติดตั้ง VS Code จาก code.visualstudio.com พร้อมติ๊ก Add to PATH
- 2 ติดตั้ง Extension: Python, Pylance, Ruff, Jupyter
- 3 **ชี้ Interpreter ไปที่ .venv ของ uv** กด Ctrl+Shift+P แล้วเลือก Python: Select Interpreter
- 4 ตรวจสอบขวาล่างต้องขึ้น Python 3.13.x ('.venv': venv)

ถ้าข้ามขั้นที่ 3 จะเจอขีดหยักแดงว่า Import could not be resolved ทั้งที่ uv add ไปแล้ว นี่คือปัญหาที่ผู้เรียนเจอบ่อยที่สุด

```
// .vscode/settings.json
{
  "python.defaultInterpreterPath":
    "${workspaceFolder}/.venv/Scripts/python.exe",
  "editor.formatOnSave": true,
  "editor.defaultFormatter":
    "charliermarsh.ruff",
  "editor.rulers": [88]
}
```

PyCharm Community

เหมาะกับโปรเจกต์ Python ขนาดใหญ่ เพราะ Refactor และ Debugger แข็งแรงกว่า ส่วน VS Code เบากว่าและดีกว่าเมื่อมีหลายภาษาในโปรเจกต์เดียว

รันโปรแกรมด้วย uv run และคำสั่งเดิมที่ควรอ่านออก

```
uv run main.py
uv run python
uv run python -m http.server 8000
uvx ruff check .
```

- uv run <ไฟล์> ใช้บ่อยที่สุดตลอดหลักสูตร
- uv run python ทดลองโค้ดสั้น ๆ แบบโต้ตอบ
- uvx <เครื่องมือ> รันครั้งเดียวโดยไม่เพิ่มเข้าโปรเจกต์

คำสั่งเดิม (pip/venv)	คำสั่งใหม่ (uv)
python -m venv .venv	uv venv
Activate.ps1	ไม่ต้องทำ (uv run)
pip install requests	uv add requests
pip uninstall requests	uv remove requests
pip list	uv pip list / uv tree
pip freeze > req.txt	uv export -o req.txt
pip install -r req.txt	uv sync

WORKSHOP 2.1

ติดตั้ง uv สร้างโปรเจกต์แรก และรัน Hello World

เป้าหมาย พิสูจน์ว่าเครื่องมือทั้งชุด uv + Python 3.13 + VS Code ทำงานครบวงจร ก่อนเข้าสู่เนื้อหาภาษาในช่วงบ่าย

```
# main.py
import sys

def main() -> None:
    print("=" * 44)
    print("  PyShop Mini")
    print("  Python Basic 2026 @ IT Genius")
    print("=" * 44)
    print(f"Python : {sys.version.split()[0]}")
    print(f"Path   : {sys.executable}")

if __name__ == "__main__":
    main()
```

- 1 ติดตั้ง uv แล้วปิด-เปิด Terminal ใหม่
- 2 uv python install 3.13 แล้ว uv init pyshop
- 3 แก้ไฟล์ main.py ตามตัวอย่าง
- 4 uv run main.py
- 5 code . แล้วเลือก Interpreter และกดรันจากใน Editor

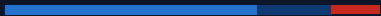
ผลลัพธ์ที่คาดหวัง เห็นบรรทัด Python 3.13.x และที่อยู่ของ Python ที่ **ต้องมีคำว่า .venv อยู่ในพาร** ซึ่งเป็นหลักฐานว่ารันอยู่ใน Virtual Environment ของโปรเจกต์จริง

03

MODULE 03 · 13:00-14:15

พื้นฐานภาษา Python

การทำตามลำดับ (Sequence) การตัดสินใจ (Selection) และการทำซ้ำ (Iteration)



โครงสร้างโปรแกรมและกฎการย่อหน้า

```
# ถูกต้อง: ย่อหน้า 4 ช่องสม่ำเสมอ
score = 85
if score >= 80:
    grade = "A"
    print("ได้เกรด A")
print("จบการทำงาน")
```

```
# ผิด: IndentationError
if score >= 80:
    grade = "A"
    print("ผิดแน่นอน")
```

ใช้ **Space 4 ช่อง** ต่อ 1 ระดับ ตามมาตรฐาน PEP 8

ห้ามผสม Tab กับ Space เป็นสาเหตุของ TabError ที่หายากที่สุด

บรรทัดที่ลงท้ายด้วย : ต้องมีบล็อทย่อหน้าตามมาเสมอ

โค้ดที่คัดลอกจากอินเทอร์เน็ตมักปน Tab กับ Space วิธีแก้เร็วที่สุดใน VS Code คือ Ctrl+Shift+P แล้วเลือก Convert Indentation to Spaces

ตัวแปร การกำหนดค่า และชนิดข้อมูลพื้นฐาน

```
product_name = "แป้นพิมพ์"    # str
price = 890.50                 # float
stock = 25                     # int
is_active = True               # bool

qty = int("12")                # str → int
price = float("890.50")
label = str(25)
flag = bool(0)                 # False
```

ชนิด	ใช้เก็บอะไร	ข้อควรระวัง
int	จำนวนเต็ม ไม่จำกัดขนาด	ใช้ _ คั่นหลักได้
float	ทศนิยม	$0.1 + 0.2 \neq 0.3$
str	ข้อความ	เปิด-ปิดชนิดเดียวกัน
bool	ค่าตรรกะ	ต้องขึ้นต้นตัวใหญ่
None	ยังไม่มีค่า	ต่างจาก 0 และ ""

ค่าที่ Python ถือว่าเป็น False (Falsy) คือ 0, 0.0, "", [], {}, (), set(), None ความรู้ที่ใช้เขียน `if not items:` แทน `if len(items) == 0:` ได้กระชับกว่า

รับค่าจากผู้ใช้และแสดงผลด้วย f-string

```
# input() คืนค่าเป็น str เสมอ
name = input("ชื่อสินค้า: ")
price = float(input("ราคา: "))
qty = int(input("จำนวน: "))
total = price * qty

print(f"{name} {qty} ชิ้น รวม {total:,.2f} บาท")
```

f-string อ่านง่ายที่สุดและเร็วที่สุด ใช้ได้ตั้งแต่ Python 3.6 ส่วน
.format() และ %-formatting ให้รู้จักไว้อ่านโค้ดเก่าเท่านั้น

f"{price:,.2f}"	→ 1,234,567.89
f"{price:>15,.2f}"	→ ชิดขวา กว้าง 15
f"{name:<10} "	→ ชิดซ้าย กว้าง 10
f"{name:^10} "	→ กึ่งกลาง
f"{ratio:.1%}"	→ 18.8%
f"{255:04d}"	→ 0255
f"{price = }"	→ price = 1234567.891

เคล็ดลับดีบั๊ก f"{ตัวแปร = }" จะพิมพ์ทั้งชื่อตัวแปรและค่าออกมาให้เลย เหมาะกับการตรวจค่ากลางทางโดยไม่ต้องพิมพ์ชื่อซ้ำสองรอบ

นิพจน์และตัวดำเนินการ

```
a, b = 17, 5

a + b    # 22
a / b    # 3.4    หาค่า float เสมอ
a // b   # 3      ปัดเศษลง
a % b    # 2      เศษจากการหาร
a ** b   # 1419857 ยกกำลัง

0 < stock <= 10    # เขียนช่วงต่อกันได้
stock == 5 and price < 1000
```

กลุ่ม	ตัวดำเนินการ	ความหมาย
เปรียบเทียบ	== != > < >= <=	คืน True/False
ตรรกะ	and or not	เชื่อมเงื่อนไข
สมาชิก	in / not in	อยู่ในลำดับหรือไม่
เอกลักษณ์	is / is not	วัตถุเดียวกันหรือไม่
กำหนดค่าย่อ	+= -= *= /= //=	x += 1 เท่ากับ x = x + 1

== เทียบค่า ส่วน is เทียบว่าเป็นวัตถุตัวเดียวกันในหน่วยความจำ กล้วยๆ คือใช้ is เฉพาะกับ None, True, False เท่านั้น นอกนั้นใช้ ==

เงื่อนไข การตัดสินใจ และ match-case

```
if stock <= 0:
    status = "สินค้าหมด"
elif stock < 10:
    status = "ใกล้หมด ควรสั่งเพิ่ม"
elif stock < 50:
    status = "ปกติ"
else:
    status = "สต็อกเพียงพอ"

# เงื่อนไขบรรทัดเดียว
label = "มีสินค้า" if stock > 0 else "หมด"
```

ใช้ if / elif เมื่อ

เงื่อนไขเป็นช่วงค่าหรือการเปรียบเทียบ มีเงื่อนไขซับซ้อนผสม and/or หรือต้องรองรับ Python เก่ากว่า 3.10

```
match command:
    case "add":
        print("เพิ่มสินค้าใหม่")
    case "list":
        print("แสดงรายการทั้งหมด")
    case "report" | "summary":
        print("สรุปรายงาน")
    case "exit" | "quit":
        print("ออกจากโปรแกรม")
    case _:
        print("ไม่รู้จักคำสั่ง")
```

ใช้ match-case เมื่อ

เทียบกับค่าที่แน่นอนหลายค่า เช่น เมนูคำสั่ง หรือต้องการแยกโครงสร้างข้อมูล (ใช้ได้ตั้งแต่ 3.10)

การทำซ้ำด้วย for และ while

for — รู้ว่าต้องวนกี่รอบ

```
for i in range(1, 6):          # 1 ถึง 5
    print(i)

for i in range(0, 101, 10):    # ก้าวทีละ 10
for i in range(5, 0, -1):      # นับถอยหลัง

for idx, p in enumerate(products, start=1):
    print(f"{idx}. {p}")

for p, price in zip(products, prices):
    print(f"{p:<12} {price:>8,} บาท")
```

break

ออกจากลูปทันที

continue

ข้ามไปเริ่มรอบถัดไป

while — รู้เงื่อนไขที่จะหยุด

```
total = 0
while True:
    text = input("ราคา (q เพื่อจบ): ")
    if text.lower() == "q":
        break          # ออกจากลูปทันที
    if not text.isdigit():
        print("ใส่ตัวเลขเท่านั้น")
        continue      # ข้ามไปรอบใหม่
    total += float(text)
```

else ของลูป

ทำงานเมื่อลูปจบโดยไม่โดน break

pass

ไม่ทำอะไร กั้นตำแหน่งไว้

Comment และมาตรฐาน PEP 8

```
# คอมเมนต์ที่ไม่มีประโยชน์
total = price * qty    # คูณราคากับจำนวน

# คอมเมนต์ที่มีประโยชน์
total = price * qty * 1.07
# รวม VAT 7% ตามข้อกำหนดฝ่ายบัญชี
```

```
uvx ruff check .    # ตรวจสอบไคส์โค้ด
uvx ruff format .   # จัดรูปแบบอัตโนมัติ
```

ไม่ต้องจำทุกกฎ ให้เครื่องมือช่วยจัดให้อัตโนมัติ

หัวข้อ	มาตรฐาน
ตัวแปร / ฟังก์ชัน	snake_case
คลาส	PascalCase
ค่าคงที่	UPPER_SNAKE_CASE
ย่อหน้า	Space 4 ช่อง
ความยาวบรรทัด	ไม่เกิน 79-88 ตัวอักษร
ช่องว่างรอบตัวดำเนินการ	มีข้างละหนึ่งช่อง
บรรทัดว่างระหว่างฟังก์ชัน	2 บรรทัด

WORKSHOP 3.1

โปรแกรมคำนวณและตรวจสอบเงื่อนไขสินค้า

โจทย เขียนโปรแกรมรับรายการสินค้าเข้ามาทีละรายการ คำนวณยอดรวมพร้อม VAT และแจ้งเตือนสถานะสต็อก โดยรวมทุกอย่างใน Module 3 เป็นโปรแกรมที่ใช้ได้จริงหนึ่งตัว

```
VAT_RATE = 0.07
LOW_STOCK_LIMIT = 10

while True:
    name = input("ชื่อสินค้า: ").strip()
    if name == "":
        break
    price = float(input("ราคา: "))
    qty = int(input("จำนวน: "))
    subtotal = price * qty

    if qty == 0:
        status = "สินค้าหมด"
    elif qty < LOW_STOCK_LIMIT:
        status = "ใกล้หมด ควรสั่งเพิ่ม"
    else:
        status = "ปกติ"

    grand_total += subtotal
```

ผลลัพธ์ที่คาดหวัง

ใส่สินค้า 3 รายการแล้วกด Enter ว่าง จะเห็นตารางสรุปพร้อม VAT จัดหน้าตรงคอลัมน์ และรายการที่น้อยกว่า 10 ขึ้นเตือนใกล้หมด

โจทยต่อยอด

เพิ่มการนับว่ามีสินค้าที่รายการอยู่ในสถานะใกล้หมด แล้วรายงานท้ายสุด และลองใส่ตัวอักษรในช่องราคาเพื่อดูว่าโปรแกรมกันไว้ได้จริงหรือไม่

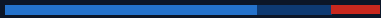
```
uv run workshop3.py
```

04

MODULE 04 · 14:15-15:00

ชนิดข้อมูลแบบกลุ่ม (Collections)

ตัวแปรเดี่ยวเก็บได้ค่าเดียว แต่งานจริงมีสินค้า 500 รายการ หรือยอดขาย 12 เดือน



4 ชนิด และการเลือกใช้ให้ถูกงาน

ชนิด	สัญลักษณ์	แก้ไขได้	ค่าซ้ำได้	เหมาะกับ
List	[]	ได้	ได้	รายการที่ต้องเพิ่ม/ลบ
Tuple	()	ไม่ได้	ได้	ข้อมูลคงที่ พิกัด ค่าที่คืนจากฟังก์ชัน
Set	{ }	ได้	ไม่ได้	ตัดค่าซ้ำ หาส่วนร่วม
Dict	{key: value}	ได้	key ห้ามซ้ำ	ข้อมูลที่มีชื่อฟิลด์กำกับ

กฎการเลือกอย่างง่าย ถ้าข้อมูลมีชื่อฟิลด์ใช้ **Dict** · ถ้าเป็นรายการที่ต้องเพิ่มลบใช้ **List** · ถ้าไม่ต้องการให้แก้ไขใช้ **Tuple** · ถ้าต้องการค่าไม่ซ้ำใช้ **Set**

List — รายการที่แก้ไขได้

```
products = ["เมาส์", "คีย์บอร์ด", "จอภาพ"]
```

```
products[0]      # ตัวแรก
products[-1]     # ตัวสุดท้าย
len(products)   # จำนวนสมาชิก
```

```
products[1:3]    # ตำแหน่ง 1 ถึง 2
products[:2]     # ตั้งแต่ต้นถึงตำแหน่ง 1
products[2:]     # ตำแหน่ง 2 ถึงท้าย
products[::-1]   # กลับลำดับทั้งหมด
```

```
products.append("ลำโพง")
products.insert(1, "แผ่นรองเมาส์")
products.extend(["สาย HDMI", "ฮับ"])
products.remove("หูฟัง")
last = products.pop()
```

```
prices.sort()           # แก่ list เดิม
new = sorted(prices)    # สร้าง list ใหม่
min(prices), max(prices), sum(prices)
```

กับดักที่พบบ่อย — การ Copy List

```
a = [1, 2, 3]
b = a          # ชี้ไปที่เดียวกัน
b.append(4)
print(a)       # [1, 2, 3, 4] ← เปลี่ยนตาม

c = a.copy()   # สำเนาจริง
```

b = a ไม่ใช่การคัดลอก แต่เป็นการตั้งชื่อเล่นให้ list เดิม ต้องใช้ a.copy() หรือ a[:] จึงจะได้สำเนาจริง

Tuple และ Set

Tuple — แก้ไขไม่ได้

```
screen_size = (1920, 1080)
info = ("PRD-001", "เมาส์", 890.0)

single = (5,)      # ต้องมีคอมมาปิดท้าย
not_tuple = (5)    # นี่คือ intธรรมดา

# แยกค่า (Unpacking) ใช้น้อยมาก
code, name, price = info

a, b = b, a        # สลับค่าใน 1 บรรทัด
```

Tuple ใช้หน่วยความจำน้อยกว่าและใช้เป็น key ของ Dictionary ได้ ในขณะที่ List ทำไม่ได้

Set — ไม่มีค่าซ้ำ

```
monday = {"เมาส์", "คีย์บอร์ด", "จอภาพ"}
tuesday = {"คีย์บอร์ด", "จอภาพ", "หูฟัง"}

monday | tuesday  # ขยายวันใดวันหนึ่ง
monday & tuesday  # ขยายทั้งสองวัน
monday - tuesday  # ขยายจันทร์แต่ไม่ขยายอังคาร
monday ^ tuesday  # ขยายวันเดียวเท่านั้น

empty = set()     # set ว่างใช้ set()
```

```
unique = list(set(emails))
unique_ordered = list(dict.fromkeys(emails))
```

Dictionary — ข้อมูลแบบ Key-Value

```
product = {  
    "code": "PRD-001",  
    "name": "เมาส์ไร้สาย",  
    "price": 890.0,  
    "stock": 25,  
}  
  
product["name"]           # อ่านค่า  
product.get("brand")      # None ไม่พัว  
product.get("brand", "ไม่ระบุ") # ค่าสำรอง  
product["brand"] = "Logitech" # เพิ่ม key  
product.update({"price": 850.0})  
removed = product.pop("brand")  
  
for key, value in product.items():  
    print(f"{key:<8}: {value}")
```

Dictionary ซ้อน List — โครงสร้างที่จะใช้ตลอดหลักสูตร

```
inventory = [  
    {"code": "PRD-001", "name": "เมาส์",  
     "price": 890, "stock": 25},  
    {"code": "PRD-002", "name": "คีย์บอร์ด",  
     "price": 1290, "stock": 8},  
]  
  
for item in inventory:  
    print(f"{item['code']} {item['name']}")  
  
total = sum(i["price"] * i["stock"]  
            for i in inventory)
```

ใน Python ก่อน 3.12 ห้ามใช้เครื่องหมายคำพูดชนิดเดียวกันซ้อนใน f-string ให้สลับใช้ f"{item['name']}"

List Comprehension — เขียนโค้ดให้กระชับ

```
[ นิพจน์ for ตัวแปร in ลำดับ if เงื่อนไข ]
```

แบบเดิม 4 บรรทัด

```
with_vat = []
for p in prices:
    with_vat.append(p * 1.07)
```

Comprehension 1 บรรทัด

```
with_vat = [p * 1.07 for p in prices]

expensive = [p for p in prices if p > 1000]

labels = ["แพง" if p > 1000 else "ปกติ"
          for p in prices]
```

รูปแบบอื่นที่ใช้บ่อย

```
# Dict Comprehension
price_map = {i["code"]: i["price"]
             for i in inventory}

# Set Comprehension
letters = {i["name"][0] for i in inventory}

# Generator Expression (ประหยัดหน่วยความจำ)
total = sum(i["price"] * i["stock"]
            for i in inventory)
```

อย่าใช้จนเกินพอดี ถ้าซ้อนเกินสองชั้นหรือยาวเกินไปบรรทัด ให้เขียนเป็น for ปกติ จะอ่านง่ายกว่า เป้าหมายคือความชัดเจน ไม่ใช่ความสั้น

WORKSHOP 4.1

จัดการชุดข้อมูลรายการสินค้าและสรุปผล

ใช้ Collections ทั้ง 4 ชนิดร่วมกันเพื่อสรุปข้อมูลคลังสินค้า ซึ่งเป็นโครงสร้างที่จะกลายเป็นแกนของ PyShop Mini

1-2

แสดงตารางสินค้า 5 รายการ แล้วสรุปมูลค่ารวม จำนวนชิ้นรวม และราคาเฉลี่ยด้วย Comprehension

3-4

หาหมวดหมู่ทั้งหมดด้วย Set แล้วกรองสินค้าใกล้เคียงและสินค้าหมดด้วย List Comprehension

5-6

สรุปมูลค่าแยกตามหมวดหมู่ด้วย Dict สะสมค่า แล้วหาสินค้าราคาสูงสุดและต่ำสุดด้วย max และ min

ผลลัพธ์ที่คาดหวัง ตารางสินค้า 5 รายการพร้อมมูลค่ารวม หมวดหมู่ 3 หมวดที่ไม่ซ้ำ รายการใกล้เคียง 2 รายการ (คีย์บอร์ด, ลำโพง) สินค้าหมด 1 รายการ (จอภาพ) และกราฟแท่งข้อความแสดงสัดส่วนมูลค่าแยกตามหมวด

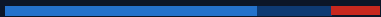
```
uv run workshop4.py
```

05

MODULE 05 · 15:00-15:30

ฟังก์ชัน (Function)

เขียนครั้งเดียว เรียกใช้ได้ทุกที่ และหนึ่งฟังก์ชันทำหนึ่งหน้าที่



สร้างฟังก์ชันและรูปแบบการรับค่า

```
def calculate_total(price, qty):  
    """คำนวณราคารวมของสินค้า"""  
    return price * qty  
  
result = calculate_total(890, 3)    # 2670  
  
# Positional / Keyword / Default  
def create_product(code, name, price,  
                   stock=0, category="ทั่วไป"):  
    ...  
  
create_product("PRD-001", "เมาส์", 890)
```

```
# *args — รับไม่จำกัดจำนวน (เป็น tuple)  
def sum_all(*numbers):  
    return sum(numbers)  
  
sum_all(10, 20, 30)    # 60  
  
# **kwargs — keyword ไม่จำกัด (เป็น dict)  
def build_report(title, **options):  
    for key, value in options.items():  
        print(f" {key} = {value}")  
  
# ลำดับบังคับ: ปกติ → *args → default → **kwargs
```

กับดักคลาสสิกที่สุดของ Python — Mutable Default Argument

```
# ผิด: list ค่าเริ่มต้นถูกใช้ร่วมกันทุกครั้ง  
def add_item(name, cart=[]):  
    cart.append(name)  
    return cart
```

```
# ถูก: ใช้ None แล้วสร้างใหม่ข้างใน  
def add_item(name, cart=None):  
    if cart is None:  
        cart = []
```

การค้นค่า ขอบเขตตัวแปร และ Type Hints

```
# คำนวณหลายค่า (เบื้องหลังคือ tuple)
def analyze_prices(prices):
    return min(prices), max(prices), \
           sum(prices) / len(prices)

low, high, avg = analyze_prices(prices)

# Guard Clause ทำให้โค้ดสั้น อ่านง่าย
def apply_discount(price, percent):
    if price <= 0:
        return 0
    return price * (1 - percent / 100)
```

LEGB Rule — Python ไล่หาชื่อตัวแปรจากในสุดออกนอกสุด Local → Enclosing → Global → Built-in

```
def calculate_total(
    price: float, qty: int,
    vat_rate: float = 0.07,
) -> float:
    """คำนวณราคารวมพร้อม VAT

    Args:
        price: ราคาต่อหน่วย (บาท)
        qty: จำนวนชิ้น
    Returns:
        ราคารวมหลังบวก VAT
    """
    return price * qty * (1 + vat_rate)
```

```
list[str]          dict[str, float]
tuple[int, int]     str | None
```

หลีกเลี่ยง `global` วิธีที่ดีกว่าคือรับค่าเข้ามาเป็นพารามิเตอร์แล้ว `return` ผลลัพธ์ออกไป

Lambda Function เบื้องต้น

ที่ใช้จริงคือใช้เป็นพารามิเตอร์ให้ฟังก์ชันอื่น โดยเฉพาะการเรียงลำดับ

```
# เรียงตามราคาจากมากไปน้อย
by_price = sorted(inventory,
                  key=lambda item: item["price"],
                  reverse=True)

# เรียงตามมูลค่าคงคลัง
by_value = sorted(inventory,
                  key=lambda i: i["price"] * i["stock"],
                  reverse=True)

# เรียงหลายเงื่อนไข
by_multi = sorted(inventory,
                  key=lambda i: (-i["stock"], i["name"]))
```

filter และ map ใช้ได้ แต่ Comprehension อ่านง่ายกว่า

```
available = list(filter(
    lambda i: i["stock"] > 0, inventory))
names = list(map(lambda i: i["name"], inventory))
```

```
available = [i for i in inventory
             if i["stock"] > 0]
names = [i["name"] for i in inventory]
```

กฎการใช้ Lambda ใช้เมื่อเป็นนิพจน์สั้น ๆ ที่ใช้ครั้งเดียว โดยเฉพาะกับ key= ถ้ามีตรรกะยาวกว่าหนึ่งบรรทัดหรือใช้ซ้ำ ให้เขียนเป็น def ที่มีชื่อจะดีกว่าเสมอ

WORKSHOP 5.1

แปลงโค้ดที่เขียนซ้ำให้เป็นฟังก์ชัน

Refactor โค้ดจาก Workshop 4.1 ให้เป็นฟังก์ชันที่นำกลับมาใช้ใหม่ได้ พร้อม Type Hints และ Docstring ครบ ซึ่งจะกลายเป็นแกนของ PyShop Mini

```
Product = dict[str, str | int | float]

def create_product(...) -> Product
def stock_value(product) -> float
def total_stock_value(products) -> float
def add_vat(amount, rate=VAT_RATE) -> float
def stock_status(product) -> str
def find_products(products, keyword) -> list
def format_row(product) -> str
def print_table(products, title) -> None
```

ผลลัพธ์ที่คาดหวัง

เห็นตาราง 3 ชุด (ทั้งหมด / ต้องสั่งเพิ่ม / ผลค้นหา) โดยใช้ฟังก์ชัน print_table() ตัวเดียวกันทั้งสามครั้ง นี่คือหัวใจของ Module 5

โจทย์ต่อยอด

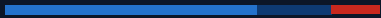
เพิ่ม apply_discount(product, percent) ที่คืนสินค้าชิ้นใหม่ พร้อมราคาลด (ห้ามแก้ dict เดิม) และ sort_products(products, by) ที่รับชื่อฟิลด์แล้วเรียงให้

06

MODULE 06 · 15:30-16:00

โมดูลและแพ็คเกจ

แยกโค้ดตามหน้าที่ จัดกลุ่มเป็น Package และยืมโค้ดคนอื่นจาก PyPI อย่างปลอดภัยด้วย uv



สร้างและเรียกใช้โมดูล

โมดูลคือไฟล์ .py หนึ่งไฟล์ ไม่มีอะไรพิเศษกว่านั้น

```
# pricing.py
VAT_RATE = 0.07

def add_vat(amount, rate=VAT_RATE):
    return amount * (1 + rate)

def apply_discount(price, percent):
    if not 0 <= percent <= 100:
        return price
    return price * (1 - percent / 100)

if __name__ == "__main__":
    print(add_vat(1000))      # 1070.0
```

```
import pricing
pricing.add_vat(1000)

from pricing import add_vat, VAT_RATE
add_vat(1000)

import pricing as pr
import pandas as pd

from pricing import *      # หลีกเลี่ยงเด็ดขาด
```

เมื่อรันไฟล์ตรง ๆ Python ตั้งค่า `__name__` เป็น `"__main__"` แต่เมื่อไฟล์ถูก import ค่าจะเป็นชื่อโมดูล บล็อกนี้จึงแยกโค้ดทดสอบออกจากโค้ดที่ให้อื่นใช้

Package และโมดูลมาตรฐานที่ควรรู้จัก

```
pyshop/
├─ pyproject.toml
├─ main.py
└─ src/
    └─ pyshop/
        ├── __init__.py
        ├── models.py
        ├── pricing.py
        ├── inventory.py
        └─ report.py
```

การวาง Package ไว้ใต้ src/ ป้องกันไม่ให้ Python เผลอ import จากไฟล์เดอร์ทำงานปัจจุบัน ทำให้มั่นใจว่าโค้ดที่ทดสอบคือโค้ดที่ติดตั้งจริง

โมดูล	ใช้ทำอะไร	จะได้ใช้เมื่อไร
os, pathlib	จัดการไฟล์และเส้นทาง	Module 7 (วันที่ 2)
sys	ข้อมูลระบบและ Python	ตรวจสอบสภาพแวดล้อม
datetime	วันที่และเวลา	บันทึกเวลาทำรายการ
math	คณิตศาสตร์ ปกติ	งานคำนวณ
random	สุ่มค่า	สร้างข้อมูลทดสอบ
json, csv	อ่าน/เขียนข้อมูล	Module 7 (วันที่ 2)
logging	บันทึกเหตุการณ์	Module 9 (วันที่ 2)

ติดตั้งไลบรารีจาก PyPI และส่งต่อโปรเจกต์

```
uv add requests
uv add "requests>=2.32"
uv add requests rich python-dotenv
uv add --dev pytest ruff
uv remove requests
uv sync          # ติดตั้งตาม uv.lock
uv lock --upgrade
uv tree          # ดูโครงสร้างแบบต้นไม้
uv export --no-hashes -o requirements.txt
```

pyproject.toml กับ uv.lock ต่างกันอย่างไร

pyproject.toml คือ **เจตนา** — คนเขียนแก้ผ่าน uv add เก็บช่วงเวอร์ชันที่ยอมรับได้

uv.lock คือ **ผลลัพธ์ที่จริงไว้** — uv สร้างให้อัตโนมัติ เก็บเวอร์ชันเป๊ะ ๆ ทุกตัวรวมตัวพ่วง ห้ามแก้ด้วยมือ

ทั้งสองไฟล์ต้องขึ้น Git เสมอ ส่วน .venv ห้ามขึ้น

เครื่องคนเขียน

```
uv add rich
pyproject.toml + uv.lock ขึ้น Git
(.venv ไม่ส่ง)
```



เครื่องเพื่อนร่วมทีม

```
git clone <repo>
uv sync
ได้ไลบรารีเวอร์ชันเดียวกันเป๊ะ
```

ไม่มีคำว่า "เครื่องผมรันไม่ได้" อีก

WORKSHOP DAY 1

PyShop Mini v1 — โปรแกรมจัดการสินค้าแบบ Console

ประกอบทุกอย่างที่เรียนมาทั้งวันเป็นโปรแกรมเมนู โดยแยกโค้ดออกเป็นโมดูลตามหน้าที่

```
pyshop/  
├─ main.py          ← เมนูและรับคำสั่งผู้ใช้  
└─ src/pyshop/  
    ├─ __init__.py  
    ├─ models.py    ← สร้างและตรวจสอบข้อมูลสินค้า  
    ├─ pricing.py   ← VAT และส่วนลด  
    ├─ inventory.py ← ค้นหา สรุป สถานะสต็อก  
    └─ report.py    ← ตารางและรายงาน
```

PyShop Mini v1 - เมนูหลัก

1. แสดงรายการสินค้าทั้งหมด
2. ค้นหาสินค้า
3. เพิ่มสินค้าใหม่
4. ปรับจำนวนคงเหลือ
5. รายการที่ต้องสั่งเพิ่ม
6. รายงานสรุปภาพรวม
0. ออกจากโปรแกรม

ข้อสังเกตสำคัญ main.py แทบไม่มีตรรกะการคำนวณเลย มีแต่การรับคำสั่งแล้วเรียกฟังก์ชันจากโมดูลอื่น นี่คือแนวคิดแยกส่วนติดต่อผู้ใช้ ออกจากตรรกะทางธุรกิจ ซึ่งทำให้วันพรุ่งนี้เราเปลี่ยนหน้าจอเป็น GUI ได้ โดยแทบไม่ต้องแก้ไฟล์ใน src/pyshop เลย

```
uv run main.py
```

DAY 1 RECAP

สรุปวันที่ 1 และเตรียมพร้อมสำหรับวันพรุ่งนี้

สิ่งที่ทำได้แล้ววันนี้

- ✓ เข้าใจจุดแข็ง ข้อจำกัด และการเลือกเวอร์ชัน Python
- ✓ ติดตั้ง uv, Python 3.13 และตั้งค่า VS Code ให้ใช้ .venv
- ✓ ตัวแปร ชนิดข้อมูล f-string เชื้อนไข และการทำซ้ำ
- ✓ เลือกใช้ List, Tuple, Set, Dict ได้ถูกงาน
- ✓ สร้างฟังก์ชันครบทุกรูปแบบพร้อม Docstring และ Type Hints
- ✓ แยกโค้ดเป็นโมดูล ใช้ uv add และ uv sync
- ✓ ได้ PyShop Mini v1 ที่แยก UI ออกจาก Logic แล้ว

พรุ่งนี้เราจะทำอะไรต่อ

File I/O	ข้อมูลย้ายไปอยู่ในไฟล์ CSV/JSON จริง ปิดโปรแกรมแล้วไม่หาย
OOP	dict ของสินค้าจะกลายเป็น class Product ที่มีเมธอดในตัว
Exception	ask_int / ask_float จะเปลี่ยนไปใช้ try/except อย่างถูกวิธี
Database	ข้อมูลย้ายจากไฟล์ไปอยู่ใน PostgreSQL
Git	โฟลเดอร์ .git ที่ uv init สร้างไว้จะได้ใช้จริง และแชร์ขึ้น GitHub
GUI	เมนูข้อความจะกลายเป็นหน้าต่างจริงด้วย Tkinter

สิ่งที่ต้องเตรียมมาพรุ่งนี้ โฟลเดอร์ pyshop ที่ทำวันนี้ (อย่าลบ) และถ้าเป็นไปได้ให้สมัครบัญชี GitHub มาล่วงหน้า